

Efficient Hardware Architecture Design of Radix-2² Fast Fourier Transform Using Coordinate Rotation Digital Computer*

Kaushik Das ^{†,§} Sambhu Nath Pradhan ^{‡,¶} and Abhishek Bhattacharjee ^{‡,||}

[†]*Department of Electronics and Communication Engineering,
GLA University, Mathura, Uttar Pradesh, India*

[‡]*Department of Electronics and Communication Engineering,
National Institute of Technology,
Agartala, Tripura, India*

[§]*kaushik.das@gla.ac.in*

[¶]*sambhupradhan.ece@nita.ac.in*

^{||}*deepbhattacharjee108@gmail.com*

Received 5 March 2024

Accepted 10 July 2024

Published 12 September 2024

The fast Fourier transform (FFT) is a widely used algorithm for computing the discrete Fourier transform (DFT) in real-time signal processing. Achieving high performance with minimal resource usage is crucial for any real-time application. This paper introduces an architecture based on coordinate rotation digital computer (CORDIC) for implementing a radix-2² FFT. Traditional FFT implementations require complex twiddle factor multiplications, which is not cost-effective in terms of hardware utilization. To address this issue, we have proposed a radix-8 CORDIC-based approach integrated with FFT computation. The proposed radix-8 CORDIC requires fewer stages than the conventional radix-2 CORDIC, enhancing computational efficiency and reducing hardware utilization. The feedforward radix-2² FFT architecture is presented where the butterflies of the FFT are computed with the help of radix-8 CORDIC. A total of three FFT architectures are developed supporting 16-, 1024- and 2048-point FFTs. The proposed FFT architectures are developed using Verilog-HDL code and implemented on a Virtex-7 field-programmable gate array (FPGA). The FPGA-based radix-2² FFT achieves a clock frequency of 323.4 MHz and utilizes minimal FPGA resources. Processing of 2048 data samples takes only 4.06 μ s by the proposed FPGA-based design. Additionally, two ASICs have been designed: the 1024-point FFT ASIC occupies an area of 1.079 mm², and the 2048-point FFT ASIC occupies an area of 1.544 mm². The proposed FFT ASICs have better-normalized area per FFT and energy per FFT than the state-of-the-art designs.

Keywords: Radix-2² FFT; CORDIC; FPGA; feedforward architectures; ASIC.

*This paper was recommended by Regional Editor Zoran Stamenkovic.

§Corresponding author.

1. Introduction

The Fourier transform is a technique of converting a time-domain signal into a frequency-domain signal by which we can analyze the frequency components present in a signal. The Fourier transform offers an important conversion tool for signal processing, which is almost impossible to tackle in the time domain. The DFT is used extensively in many digital signal processing (DSP) applications. Due to the high computational complexity in the case of DFT, the hardware implementation of DFT is not an easy task. The fast Fourier transform (FFT) is an algorithm by which the exact same results as that of DFT can be achieved by using fewer operations as compared to DFT. Till date, many FFT algorithms were proposed by the authors such as radix- r , split-radix and radix- r^k algorithm.^{1,2} The FFT algorithm proposed by Cooley and Tukey is widely used to compute N -point DFT when N can be expressed as the power of two. The DFT has a computational complexity of $O(N^2)$, whereas the FFT algorithm reduces the computational complexity of the DFT to $O(N \log_2(N))$.

Coordinate rotation digital computer (CORDIC) is an iterative process and needed only simple add and shift operation in order to perform many complex calculations like logarithmic, hyperbolic, trigonometric, real and complex multiplication, square root findings and many more.³ With all these applications, CORDIC becomes an important part of many DSP-based systems. The number of iterations required in the case of traditional radix-2 CORDIC algorithm is more, which limits the use of radix-2 CORDIC for high-speed design. To tackle this issue, several high radix CORDIC algorithms are proposed in order to reduce the iteration number and due to which the latency time of the design is also reduced.^{4,5} For m -bit precision, the conventional radix-2 CORDIC algorithm takes m -iterations. Whereas, the radix-4 CORDIC algorithm requires $m/2$ iterations.^{5,6} CORDIC is basically a rotation-based algorithm that can compute complex mathematical functions by rotating the vector in a two-dimensional plane. The CORDIC can be used in mainly two modes which are rotation and vectoring modes.⁷

In Aggarwal *et al.*,⁸ the authors have presented a reconfigurable CORDIC algorithm, their proposed design has a relatively less area complexity. The design presented in Shukla and Ray⁹ needs very less iterations for the CORDIC implementation. However, the main disadvantage of this design is the high hardware complexity. A hardware architecture design for the radix-8 CORDIC was presented in Changela *et al.*¹⁰ The selection function for the radix-8 CORDIC is difficult to implement due to the fact that it is not an integer power of two. There are nine directions of rotations in radix-8 CORDIC, and it is time-consuming to select a suitable angle out of the nine elementary angles. The authors have used the SRT division method to calculate the selection function.¹⁰

The problem with using the SRT division method is that it increases the latency of the design.¹¹ The authors have used the radix-8 SRT division method to reduce the latency. However, choosing the quotient digits gets more complicated as the

radix number increases. In our radix-8 CORDIC, we solved this problem by pre-computing all the directions of rotations for the given input angle. In Tang and Xu,¹² a field-programmable gate array (FPGA)-based architecture for the noniterative version of radix-8 CORDIC was presented. The FPGA-based design of a reconfigurable radix-2 CORDIC that works in the circular, linear, and hyperbolic coordinate systems is presented in Mohamed *et al.*¹³ The main disadvantages of the design are that the design consumes more hardware resources and has a low frequency of operation.

Till date, many parallel and pipelining hardware architecture designs for the FFT algorithm were presented. The FFT designs with parallel architectures were presented in Liu *et al.*¹⁴ and El-Motaz *et al.*¹⁵ Different algorithms are found in the literature that are widely used to solve various mathematical problems.^{16,17} To meet superior performance and real-time requirements, many pipelined hardware architectures for the FFT were proposed.^{18,19} The pipelined architectures are divided into two main types, namely, feedback architecture and feedforward architecture. The feedback architectures are further divided into two main types, single-path delay feedback (SPDF) and multi-path delay feedback (MPDF).^{18,20} The SPDF architecture can continuously process data with one sample per clock cycle. On the other hand, MPDF architecture can process many samples in each clock cycle. The multi-path delay commutator (MPDC) or feedforward architectures are also capable of processing many samples per clock cycle.¹⁸

In Ref. 21, a distributed arithmetic (DA)-based radix-2 FFT that operates on complex binary number system (CBNS) was presented. Compared to the design proposed in Ref. 21, our design needs very few LUTs and the execution time of our design is also very less. A low-cost and power-efficient processor design for FFT was proposed in Kang *et al.*²² The SPDF architecture for the FFT which supports 46 different sizes of FFT was presented in Shih *et al.*²³ In order to reduce the number of operations in an FFT ASIC, the authors have used the concept of hashing and lookup table (LUT).²⁴ The ASIC can process only 128 points and has relatively more execution time. Ingemarsson *et al.* presented a single-stream feedforward (SFF) architecture for the FFT.²⁵ The design's two main drawbacks are more hardware utilization and high execution time.

FPGA and ASIC implementation of memory-based FFT was proposed by Wang *et al.*²⁶ Their proposed ASIC was operated only with a 150 MHz clock frequency. The authors have proposed a hardware architecture design for the pipelined FFT, which includes serial and parallel split and merge processing butterfly units.²⁷ The proposed method uses a lot of FPGA resources. Moreover, for multiplication operation, the authors used an extra multiplier and divider circuit to compensate for the fractional part, which further will increase the area and computation time of the design.²⁷ A CORDIC-based implementation of FFT is presented in Inguva and Seventiline.²⁸ The authors have used the canonical signed-digit (CSD) approach and the Hcub method for the reduction of hardware resources. Still, the design consumes more hardware resources, and the operating frequency of the design is also less.²⁸

There are many works that have been done on FFT to reduce the area and power of the FFT design. In Ayinala *et al.*,²⁹ the authors have used the folding transformation and register minimization technique to reduce the power and area of the FFT design. The authors have presented a mixed SPDF and SPDC pipelined radix-2 FFT architecture to reduce the complex multipliers as well as complex adders requirement. However, the disadvantage of the design is the high latency and low throughput.³⁰ In Wang *et al.*,³¹ a mixed-decimation MPDF method for the radix-2^k FFT algorithm was presented. The design presented by the authors has a reduced computational delay and has a low-memory requirement.³¹

A summary of recent related works found in the literature is presented in Table 1. In the last two decades, several FPGA and ASIC-based architectures for the FFT algorithm were presented. However, in almost all the designs, two common problems are seen which are more hardware requirement and high-execution time. In this paper, we have presented an FPGA- and ASIC-based design for radix-2² FFT. The butterflies of the FFT are computed by using a radix-8 CORDIC. With the help of radix-8 CORDIC, we have reduced the iteration stages in the CORDIC to $m/3$ (for m -bit precision). The radix-8 CORDIC has nine directions of rotation and due to which the delay of the design increases. To solve this problem, we have precomputed all the directions of rotation.

The proposed design has high throughput, less execution time and less hardware requirement as compared to the existing designs. The feedforward architecture of the FFT can process four samples in each clock cycle, due to which the throughput of the design is increased. The pipeline architecture has been used with parallel processing in order to achieve a faster implementation. The proposed FPGA-based 1024-point FFT architecture has an area saving of 17.73% and 1.49 times faster as compared to

Table 1. Summary of some recent related works found in the literature.

Year	Key contributions
2023	A processor design for the FFT that supports 60-mode was proposed in Ref. 32. In this design, the authors introduced a conflict-free memory access scheme using fusion shift and a hopping-based approach to cut down on cycle costs for data allocation. The operating clock frequency of the design is 1 GHz and has a throughput of 3.92 GS/s ³²
2022	The authors have proposed an FFT processor based on the split-radix algorithm. ³³ To reduce the complexity of the design, the authors presented a data flow graph, which requires no non-trivial twiddle factors in the odd stages of the algorithm. Their 256-point FFT architecture achieved a clock frequency of 877 MHz and a power consumption of 597.94 mW ³³
2023	In Ref. 34, an FFT architecture for the processing of real-valued and complex-valued signals is proposed. The architecture can process two samples simultaneously. The authors designed an ASIC using 65 nm technology, and the ASIC has an area of 0.97 mm ² ³⁴
2024	A serial commutator pipelined FFT architecture was presented in Ref. 35. In this design, the authors basically tried to reduce the number of multiplexers in the circuit. By this, they were able to reduce the overall hardware consumption of the FFT design ³⁵
2024	In Ref. 36, the authors presented a CBNS based FFT ASIC. By using the CBNS representation, the authors have reduced the hardware cost as well as the computational time. The ASIC has a core area of 1.11 mm ² and was designed using a 180 nm cell library

the best available design. The proposed ASIC has the least normalized area and energy per FFT compared to the available design. Also, the proposed ASIC can be operated at a faster rate.

The remainder of the paper is arranged as follows. Sections 2 and 3 cover the theory of the radix-2² FFT and the radix-8 CORDIC, respectively. Section 4 presents the details of the proposed radix-8 CORDIC's hardware architecture. The feedforward architecture of the proposed four-parallel radix-2² FFT using radix-8 CORDIC as the butterfly unit is presented in Sec. 5. Section 6 includes the results and discussion. Section 7 concludes.

2. Radix-2² FFT Algorithm

For an input sequence $x[n]$, the N -point DFT for the sequence can be given as follows:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, 1, \dots, N-1, \quad (1)$$

where $W_N^{kn} = e^{-j(\frac{2\pi}{N})kn}$ is known as the twiddle factor. The FFT is an algorithm by which the DFT can be computed efficiently. The radix-2 FFT algorithm decomposes Eq. (1), into even ($k = 2m$) and odd ($k = 2m + 1$) number of samples as shown in Eqs. (2) and (3), respectively.

$$X[2m] = \sum_{n=0}^{\frac{N}{2}-1} \left(x[n] + x \left[n + \frac{N}{2} \right] \right) W_{N/2}^{mn}, \quad (2)$$

$$X[2m+1] = \sum_{n=0}^{\frac{N}{2}-1} \left(x[n] - x \left[n + \frac{N}{2} \right] \right) W_{N/2}^{mn} W_N^n, \quad (3)$$

where N is an integer power of 2 and $0 \leq m \leq N/2 - 1$. Based on the simple architecture of radix-2 algorithm, the radix-2² algorithm was developed and it requires $\log_4 N - 1$ number of iteration stages. This is same as the iteration stages required by a radix-4 FFT. The radix-2² algorithm requires very few multipliers, which makes the architecture simple and suitable for pipeline implementation of FFT. The generalized expression of the radix-2² algorithm can be defined by the following equation³⁷:

$$X[k_1 + 2k_2 + 4k_3] = \sum_{n_3=0}^{\frac{N}{4}-1} [G(k_1, k_2, k_3)] W_{\frac{N}{4}}^{k_3 n_3}, \quad (4)$$

where

$$G(k_1, k_2, k_3) = \left[\left(x[n_3] + (-1)^{k_1} x \left[n_3 + \frac{N}{2} \right] \right) + (-j)^{(k_1+2k_2)} \right. \\ \left. \times \left(x \left[n_3 + \frac{N}{4} \right] + (-1)^{k_1} x \left[n_3 + \frac{3N}{4} \right] \right) \right] W_N^{n_3(k_1+2k_2)}, \quad (5)$$

$$G(k_1, k_2, k_3) = [X + (-j)^{(k_1+2k_2)} Y] W_N^{n_3(k_1+2k_2)}. \quad (6)$$

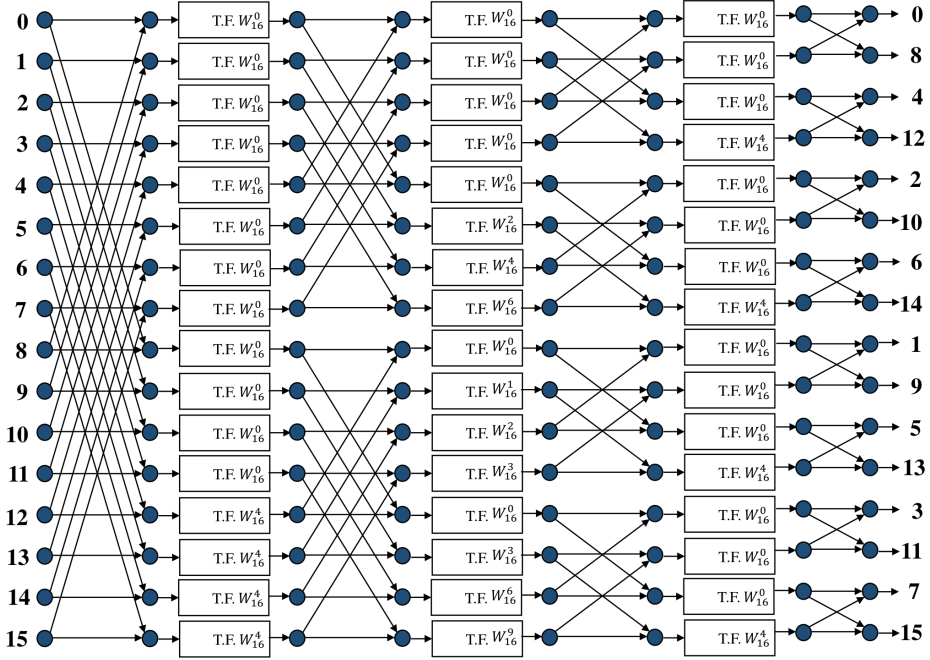


Fig. 1. Flow graph representation of 16-point radix-2² DIF FFT algorithm (TF=twiddle factor).

In Eq. (6), X and Y are the two radix-2 FFTs and $G(k_1, k_2, k_3)$ consisting of X and Y is also a radix-2 FFT with an extra complex multiplier. The flow graph representation of a 16-point radix-2² DIF FFT algorithm is shown in Fig. 1.

3. Radix-8 CORDIC Algorithm

The main reason behind the use of radix-8 CORDIC is that the number of iterations is decreased if we use radix-8 CORDIC instead of radix-2 CORDIC. In order to compute the total number of rotations for the precision of m -bit, the radix-2 algorithm requires m iterations, whereas the number of iterations is decreased to $m/3$ if we use radix-8 CORDIC. The CORDIC algorithm is based on a two-dimensional xy -plane vector rotation which is shown in Fig. 2. As shown in Fig. 2, the two-dimensional vector $B_n(x_n, y_n)$ with angle θ is rotated anticlockwise by an amount of angle β . The rotated vector B_{n+1} has coordinates (x_{n+1}, y_{n+1}) and angle $(\theta + \beta)$. The iterative equations of the radix-8 CORDIC algorithm are given as follows:

$$x_{n+1} = x_n - \sigma_n r^{-n} y_n, \quad (7)$$

$$y_{n+1} = y_n + \sigma_n r^{-n} x_n, \quad (8)$$

$$z_{n+1} = z_n - \tan^{-1}(\sigma_n r^{-n}), \quad (9)$$

where r denotes the radix of the CORDIC (for radix-8, $r = 8$), $\sigma \in \mathbb{Z}$, $-4 \leq \sigma \leq 4$ denotes the rotational direction for each iteration, z_0 is the angle of rotation, x_0 and y_0 are the initial vector coordinates. Similarly, x_{n+1} and y_{n+1} are the vector

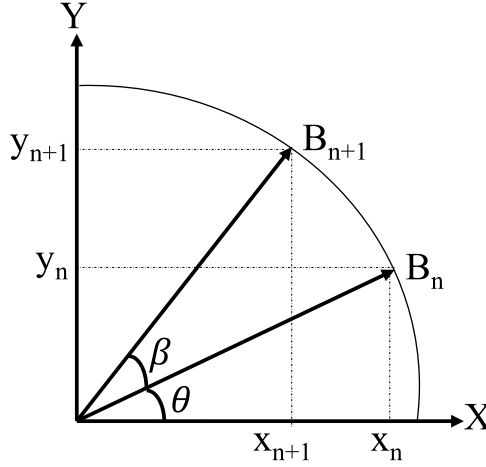


Fig. 2. Two-dimensional vector rotation of the CORDIC.

coordinates obtained by employing $(n + 1)$ micro-rotation and z_{n+1} is the angle to be rotated. The final coordinates obtained should be multiplied by a scale factor K^{-1} and can be given by

$$K^{-1} = (1 + \sigma_n^2 8^{-2n})^{-\frac{1}{2}} = (1 + \sigma_n^2 2^{-6n})^{-\frac{1}{2}}. \quad (10)$$

In case of radix-8 CORDIC, the scale factor depends on the sequence σ_n and therefore it is variable. The value of K varies from 1 to 4.41 in case of radix-8 CORDIC.

4. Proposed Radix-8 CORDIC Architecture

In this paper, a hardware architecture for a parallel CORDIC algorithm with high performance and low latency is presented in order to compute the multiplication of the complex twiddle factor. In the case of radix-2 CORDIC, the latency is high. In Antelo *et al.*,⁵ the authors have reduced the latency of radix-2 CORDIC with the help of redundant arithmetic (RA) and radix-4 number system. However, compared to radix-2 arithmetic, the delay required by the redundant radix-4 arithmetic (RR4A) is more in order to find out the direction of rotation iteratively. The reason is that in the case of radix-4 CORDIC, five rotation direction values are present and it takes more time to select among them. However, in the case of our proposed radix-8 CORDIC for the given input angle, we have precomputed all the directions of rotations in order to solve this problem.³⁸

As shown in Fig. 3, the σ prediction block and x/y path are the two basic blocks of the proposed CORDIC rotator. For the given target angle θ , σ_n values are pre-computed by the σ prediction block. For every clock cycle, the σ_n values are sent through the different stages of the x/y path in a sequential manner.

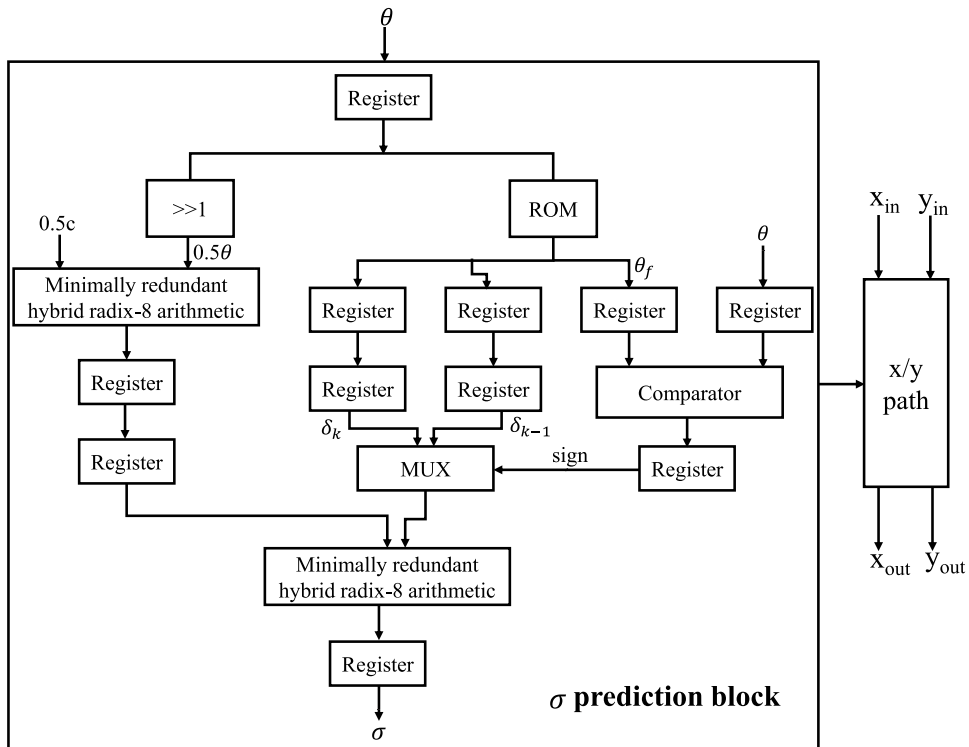


Fig. 3. Block diagram representation of the σ prediction block with the x/y path.

4.1. Precomputation of the direction of rotation (σ precomputation)

The main reason for high latency in the case of the CORDIC-based design is that in these designs, the direction of rotations is calculated in an iterative manner. In our case, latency is reduced by using the σ precomputation module. For any given input angle, the σ precomputation module calculates the direction of rotation before the start of the rotation in the x/y data path. The values are calculated by using the following formula³⁸:

$$\sigma = b + 0.5c + \varepsilon_0 \text{sign}(\theta) + \delta, \quad (11)$$

where $b = 0.5\theta$, $c = 2 - \sum_{i=0}^{\infty} (2^{-i} - \tan^{-1}(2^{-i}))$ is a constant, $\delta = \sum_{i=1}^{n/3} (\varepsilon_i \sigma_i)$ denotes the total amount of offset and in the i th iteration, the partial offset is denoted by ε_i and is given by $\varepsilon_i = (2^{-i} - \tan^{-1}(2^{-i}))$. By using simple addition, σ can be calculated from θ and the same is realized by using minimally redundant radix-8 arithmetic (RR8A),³⁹ which is shown in Fig. 3. The reason behind the use of RA is that the latency of the design can be reduced with the help of it. The flow graph to compute the minimally RA is shown in Fig. 4.

The values of δ for a given range and precision of the angle θ are precomputed and are stored in a ROM. θ_f is the reference angle and each location of the ROM is

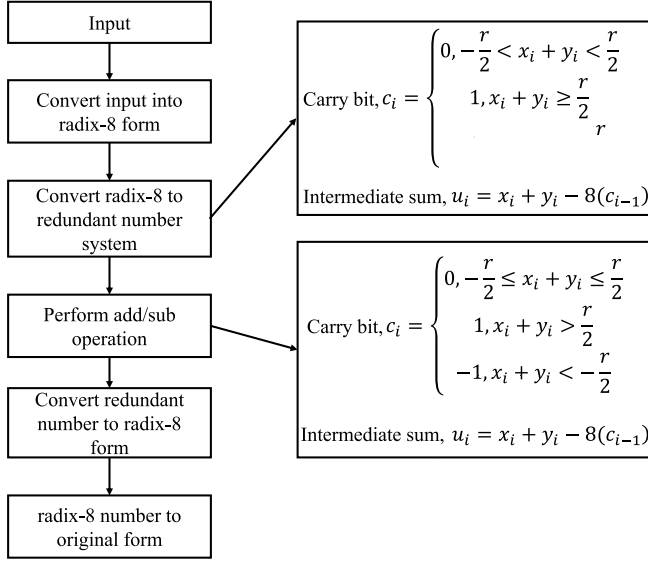


Fig. 4. Flow diagram to compute minimally RA.

comprised of a reference angle, the previous reference angle and two offset values (δ_k, δ_{k-1}) . A ROM with size $2^{\lceil m/3 \rceil} \times 3m$ is considered in order to store the values of δ for the input angle in the range of $(-\frac{\pi}{2}, \frac{\pi}{2})$ with a precision of m-bit.³⁸ The θ_f represents the angle corresponding to each of the δ and calculated from the summation of angles for the starting $m/3$ micro-rotations. In order to get a correct offset value, θ_f value should be less than the target angle value. To choose between the two offset values stored in the ROM, a 2:1 multiplexer is used which is dependent on the select signal S . The θ values are compared with θ_f values with the help of the comparator and the select signal S is generated, where $S = 0$, when $\theta > \theta_{\text{ref}}$ for selecting δ_k and $S = 1$, when $\theta < \theta_{\text{ref}}$ for selecting δ_{k-1} .

The $0.5c$ with 2's complement binary representation has been converted to a minimally redundant radix-8 (mRR8) format which is having a delay as that of an XOR gate.³⁹ It produces the redundant representation of $0.5c$ with a digit set $\{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$. The radix-8 complement representation of ϵ_0 , 0.5θ and δ_{ROM} is added with the mRR8 format of $0.5c$ by using three minimally redundant hybrid radix-8 adders in order to obtain $\sigma \in \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$. Each of which is having a delay of $2t_{\text{FA}}$, t_{FA} is the delay of one full adder. The θ is right shifted by one to obtain 0.5θ .

4.2. Scale factor

The scale factor is not a constant for radix-8 CORDIC. The final coordinate values of the rotated vector are scaled by a scale factor given by Eq. (10). Here, we have taken the first $(m/8 + 1)$ iterations in order to compute the K^{-1} and after these iterations,

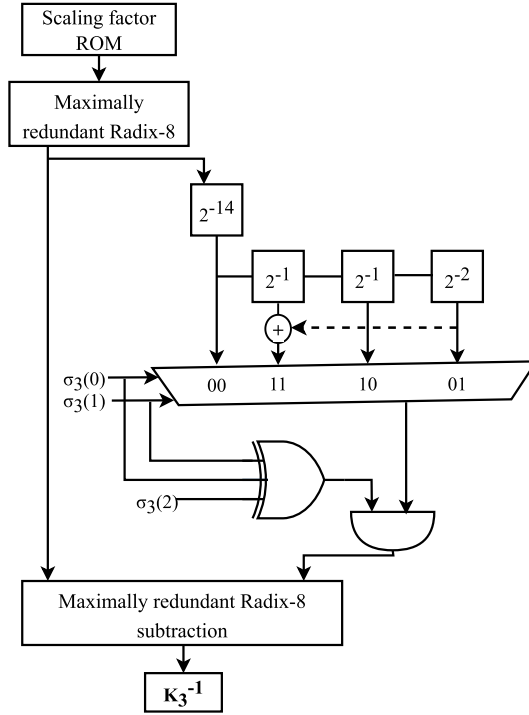


Fig. 5. Computation of scale factor for a precision of 16-bit.

the value of the scale factor tends towards unity. In this design, parallel to micro-rotations evaluation we can obtain the scale factor. For each iteration n , σ_n^2 can take any one value out of 0, 1, 4, 9 and 16. For m -bit precision, $5^{\lfloor \frac{m}{8} \rfloor + 1} \times m$ possible numbers of scale factors need to be stored in a ROM. For $n \geq \lfloor \frac{m}{8} \rfloor + 1$, the ROM size can be decreased by performing the approximation of the scale factor with the first two terms of its Taylor series expansion (TSE), when the value of m is $m \leq 16$ (the details about the TSE can be found in Antelo *et al.*⁵).

For $n \geq \lfloor \frac{m}{12} \rfloor + 1$, the approximation of the scale factor can be done by the first three terms of its TSE, when m is $m > 16$.⁵ For precision of 16-bit, in order to store 5^3 scale factors we need a 125×16 ROM. With the help of a circuit which is shown in Fig. 5, we can compute the scale factors for the next iteration. Thus parallel to the computation of the x/y coordinates we can compute the values for the scale factors. For the first two iterations, the values of the scale factor K_2^{-1} can be obtained as

$$K_n^{-1} = (1 + 8^{-2n} \sigma_n^2)^{-\frac{1}{2}}, \quad (12)$$

$$K_2^{-1} = \prod_{n=0}^2 K_n^{-1}. \quad (13)$$

By performing add and shift operations on the data present in the ROM, the scale factor for the next iteration K_3^{-1} is calculated. This is achieved by taking the first

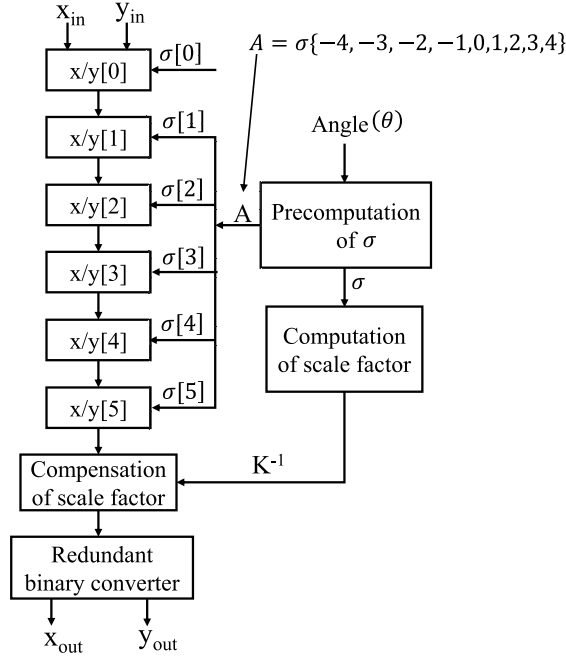


Fig. 6. The block diagram represents the x/y path of the proposed CORDIC design (for 16-bit).

two terms of their TSE as

$$K_3^{-1} = K_2^{-1} \left(1 - \frac{1}{2} 2^{-18} \sigma_3^2 \right)^{-\frac{1}{2}}. \quad (14)$$

To get the proper coordinate values for x and y , the K^{-1} values are multiplied with the x and y values. In our design, in the multiplier, we have reduced the number of nonzero digits in order to decrease the number of partial products. The scale factor is represented by using canonical signed digit because it is helpful for the reduction of nonzero digits.⁴⁰ In Fig. 6, we have shown the x/y data path of the proposed radix-8 CORDIC architecture.

4.3. The x/y data path

For the precision of m -bit, the proposed architecture needs $m/3$ stages. Every stage of the x/y data path needs a 2:1 multiplexer, an AND gate and a maximally redundant radix-8 adder or subtractor in order to realize Eqs. (7) and (8), which is shown in Fig. 7. The rounding error and the angle approximation error (AAE) affect the computational accuracy of the x and y coordinates. The AAE is due to the representation of rotation angle in quantized form.

After every m iteration, truncation of intermediate outputs produces a rounding error of $\log_2 m$ bits. For the implementation of the radix-8 CORDIC algorithm, some additional $\log_2 m$ bits known as guard bits also need to be considered.⁴ Therefore, for

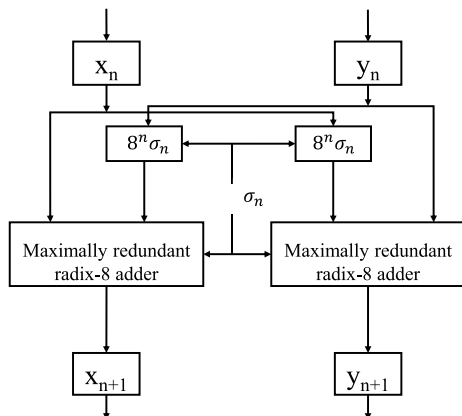


Fig. 7. The block diagram represents a microrotation in x/y path.

16-bit precision, we have taken a data bus of 20-bit wide in order to design the x/y data path. The scaled x and y coordinates are converted to a radix-2 binary representation and the converters are designed using multiplexer cells with tree-based approach. The redundant to binary converter is designed with the help of this approach having a delay of only $0.5(\lceil \log_2 m \rceil + 3)t_{FA}$.

5. The Architecture of the Proposed Radix-2² Feedforward FFT

In this proposed design, we have replaced the complex rotators with the help of radix-8 CORDIC. The proposed DIF FFT is designed using radix-2² feedforward architecture. The radix-2² feedforward architecture for the computation of 16-point and 64-point FFT is shown in Fig. 8. The R2 block, as shown in Fig. 8, denotes the radix-2 butterfly structure having complex inputs and outputs. The R2_COR1 block consists of radix-2 butterflies utilizing a single radix-8 CORDIC unit, whereas the R2_COR2 block comprises radix-2 butterflies with two radix-8 CORDIC units instead of complex rotators. The steps involved in order to compute the radix-8 CORDIC are explained in Sec. 4. The trivial rotations' (TR) block is responsible for the computation of all the TR. The TR can be obtained by only changing the sign of the data and/or by doing the interchange between imaginary and real parts.

The internal block diagram representation of the R2_COR1 and R2_COR2 is shown in Figs. 9(a) and 9(b), respectively. The size of the delay line is dependent on the total delay generated by the radix-8 CORDIC unit. The order of data at stages 1–4 of Fig. 8(a) is shown in Fig. 10. The proposed radix-2² FFT can process four samples per clock cycle. As shown in Fig. 10, the samples with indices $\{0, 8, 4$ and $12\}$ arrive at the different terminals at the same clock cycle. Whereas the samples having the indices $\{0, 1, 2$ and $3\}$ arrive at the same terminal at the consecutive clock cycles.

Efficient Hardware Architecture Design of Radix-2² FFT Using CORDIC

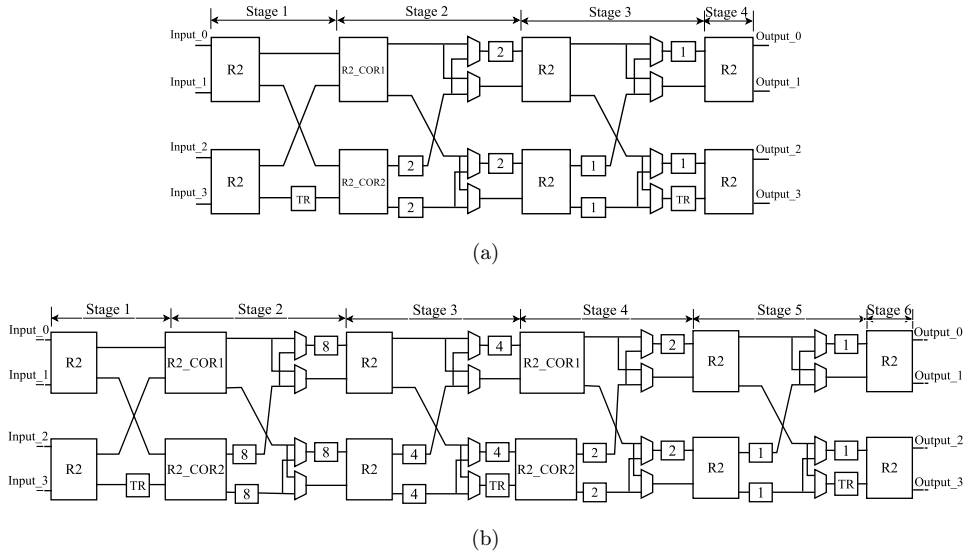


Fig. 8. Feedforward FFT design using CORDIC (a) for 16-point and (b) for 64-point.

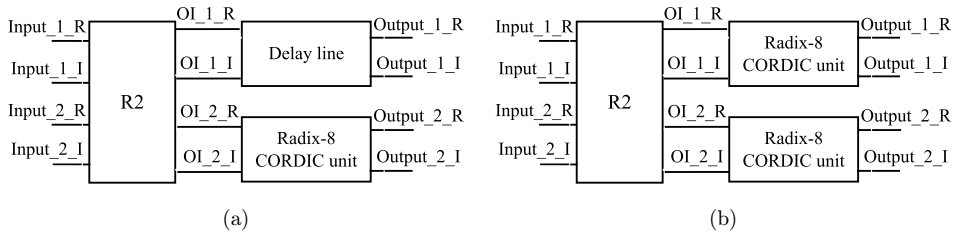


Fig. 9. Block diagram of (a) R2COR1 and (b) R2COR2 block.

n3 = 2 n3 = 1 n3 = 0			
3	2	1	0
11	10	9	8
7	6	5	4
15	14	13	12

(a)

n3 = 2 n3 = 1 n3 = 0			
3	2	1	0
7	6	5	4
11	10	9	8
15	14	13	12

(b)

n3 = 2 n3 = 1 n3 = 0			
9	8	1	0
11	10	3	2
13	12	5	4
15	14	7	6

(c)

n3 = 2 n3 = 1 n3 = 0			
12	8	4	0
13	9	5	1
14	10	6	2
15	11	7	3

(d)

Fig. 10. Data flow representation for each stage of Fig. 8(a): (a) stage 1 (b) stage 2 (c) stage 3 and (d) stage 4.

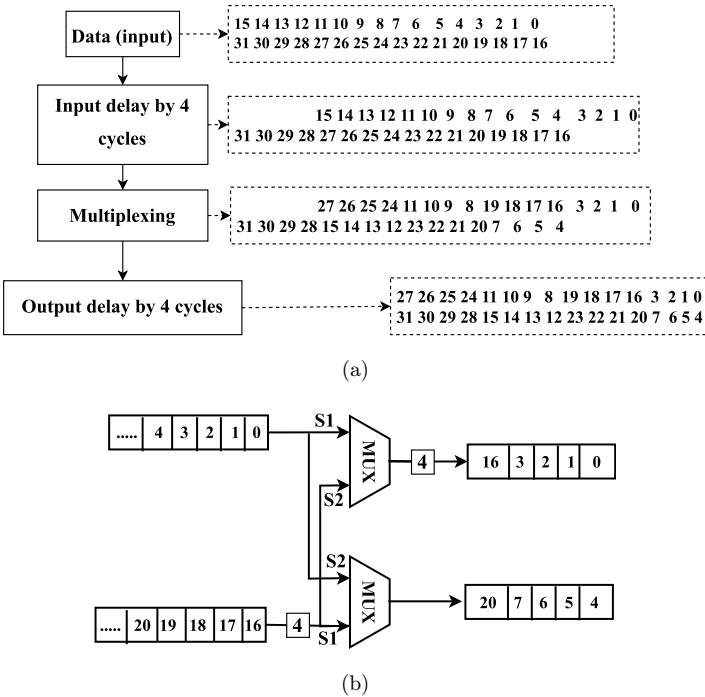


Fig. 11. The architecture of the input scheduler used in Fig. 8.

At the first phase (S1), the upper input enters the upper MUX and the input that is delayed enters the lower MUX, for the four cycles. The multiplexers switch to the second phase (S2) after the four cycles and in that phase, the lower MUX takes the data from the upper input and produces the output. In the same phase, the upper MUX only receives the data from the lower input. The lower output is separated by a distance of four from the data flowing out of the upper output delay line.

In Fig. 11(a), we have shown the example of a data flow using the input scheduler architecture shown in Fig. 11(b). The architecture basically consists of multiplexers and buffers to perform the required operation. As shown in Fig. 11(b), two incoming data are separated by 16 in this example, whereas any two output data are separated by four. In order to delay the input by 4 clock cycles, an input delay line with four shift registers is used.

6. Results and Discussion

The architecture of the proposed CORDIC-based radix-2² FFT is designed using Verilog-HDL and the Verilog code for the design is written in Xilinx Vivado design suite software. After the radix-2² FFT code is written using Verilog-HDL, the code is tested by giving some test input. The output of the Vivado simulator for 16-point

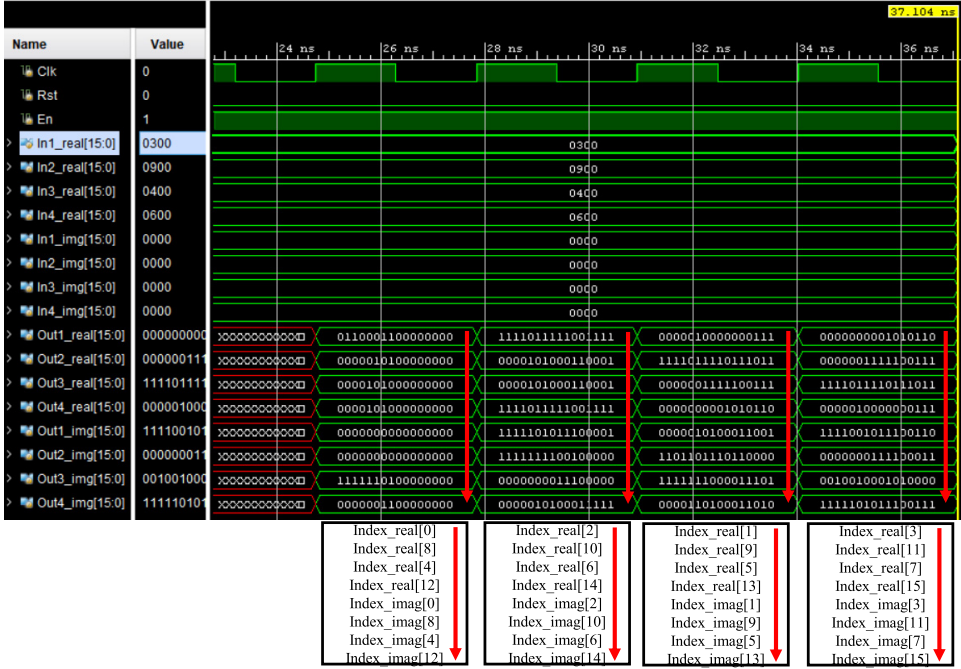


Fig. 12. The output of the Vivado simulator for 16-point FFT.

FFT is shown in Fig. 12. After that, the FFT code is synthesized and implemented by targeting a Virtex-7 FPGA (VC707 Evaluation Platform xc7vx485tffg1761-2). When the FPGA-based results for the radix-2² FFT using CORDIC are obtained as desired. The code is taken into the Cadence Genus tool and synthesis operation is done to obtain a gate level netlist file.

During the synthesis process, SCL 180 nm cell library is used. The synthesized constraint file and the synthesized module files for the computation of the FFT are taken into the Cadence Innovus tool and all the types of placement and routing (PnR) operations are done. To check the accuracy of the proposed FFT design, the FFT is also calculated for the same inputs by using the MATLAB inbuilt function “fft”. The outputs of the proposed design and the MATLAB are shown in Table 2.

For the proposed 1024-point FFT, a complex signal $y(t)$ is taken as an input, which is given by $y(t) = 0.8 \times \sin(f_1 \times \pi \times t) + i \times 0.5 \times \sin(f_2 \times \pi \times t)$. The plot of the signal $y(t)$ is shown in Fig. 13. In our experiment, we have taken the values of $f_1 = 100$ Hz and the values of $f_2 = 200$ Hz. The input signal $y(t)$ is generated and sampled. The samples are converted into hexadecimal format and saved in a .dat file, and linked to the Verilog code of our proposed radix-2² FFT design.

Figure 14 shows the FFT spectrum of the signal $y(t)$ obtained by using the MATLAB function (using the inbuilt function “fft”) and using our proposed design. In the case of our proposed design, fixed-point arithmetic (eight integer bits and eight

Table 2. Output comparison of the proposed design with MATLAB for 16-point FFT.

Index	Input	MATLAB output	Post-implemented HDL output
0	8	99.0000 + 0.0000i	99.0000 + 0.0000i
1	5	4.0288 + 5.1014i	4.0273 + 5.0977i
2	0	−8.1924 − 5.1213i	−8.1914 − 5.1211i
3	14	0.3392 − 13.1018i	0.3359 − 13.1016i
4	5	10.0000 − 3.0000i	10.0000 − 3.0000i
5	12	3.9034 − 1.8886i	3.9023 − 1.8867i
6	3	10.1924 + 0.8787i	10.1914 + 0.875i
7	4	−8.2715 + 36.3146i	−8.2695 + 36.3125i
8	8	5.0000 + 0.0000i	5.0000 + 0.0000i
9	2	−8.2715 − 36.3146i	−8.2695 − 36.3125i
10	9	10.1924 − 0.8787i	10.1914 − 0.875i
11	1	3.9034 + 1.8886i	3.9023 + 1.8867i
12	10	10.0000 + 3.0000i	10.0000 + 3.0000i
13	3	0.3392 + 13.1018i	0.3359 + 13.1016i
14	9	−8.1924 + 5.1213i	−8.1914 + 5.1211i
15	6	4.0288 − 5.1014i	4.0273 − 5.0977i

fractional bits) is used, due to which some truncation and rounding errors are observed in our case.

The output of the Verilog code of the radix-2² FFT is saved in a .dat file, signal-to-noise ratio (SNR) and peak SNR (PSNR) values are calculated for the HDL code output and MATLAB function output. The SNR is defined as the ratio of input signal power to the power of the background noise. The PSNR is defined as the ratio

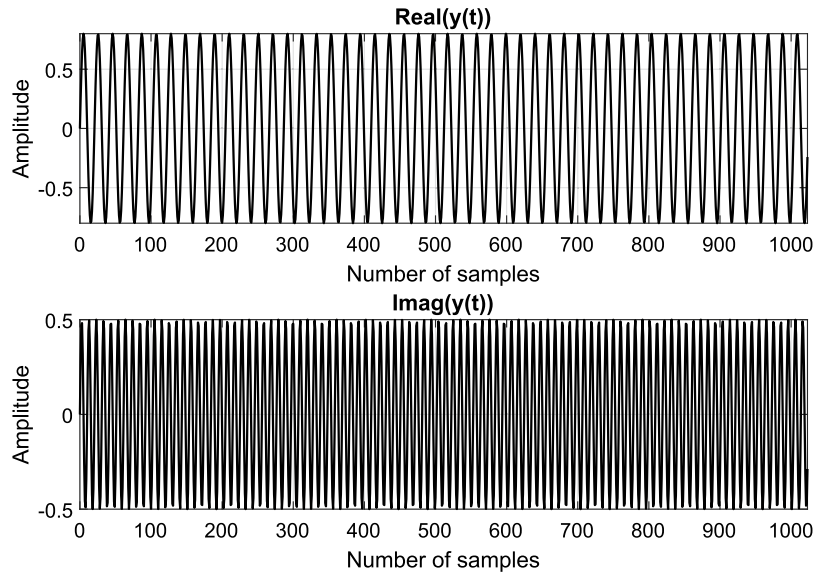


Fig. 13. The plot of the real and imaginary part of the signal $y(t)$.

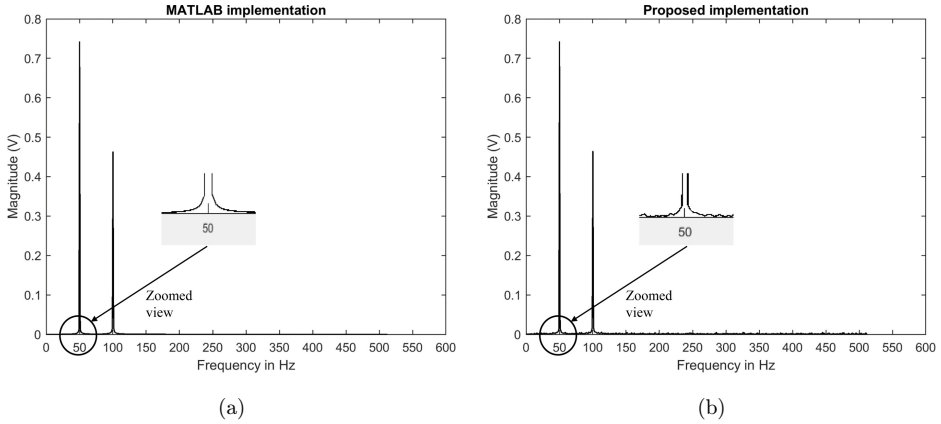


Fig. 14. FFT spectrum (a) MATLAB implementation and (b) of the proposed HDL-based design.

of an input signal's greatest power to the mean sum of all squared differences. The PSNR is given by the following formula:

$$\text{PSNR} = 10 \times \log \left(\frac{X_{\max}^2}{\frac{1}{N} \sum_{i=1}^N (X_i - Y_i)^2} \right), \quad (15)$$

where X_i and Y_i are the outputs of the proposed design and the output of the MATLAB implementation, respectively. $X_{\max}$ is the maximum value (amplitude) of the given input. In Table 3, we have shown the comparison of our proposed HDL implementation with the MATLAB implementation by considering two parameters such as SNR and PSNR. As it can be seen from Table 3, due to the reason of quantization and truncation error for 1024 point FFT, we have a loss of 3.37 dB and there is a loss 2.08 dB for 2048 point FFT. The PSNR values are also shown in Table 3.

As shown in Table 4, the proposed radix-8 CORDIC design is compared with some existing designs in terms of FPGA resources utilized and maximum clock frequency. As compared to the Xilinx CORDIC IP,⁴¹ Aggarwal *et al.*,⁸ Tang and Xu¹² and Mohamed *et al.*¹³ in our case, we have an advantage of higher clock frequency. Also, the proposed design consumes 17.17% fewer FPGA resources than Xilinx CORDIC IP. As compared to the design proposed in Mohamed *et al.*,¹³ there

Table 3. SNR values of our proposed HDL implementation and MATLAB implementation.

Point of FFT	Proposed SNR (dB)	MATLAB SNR (dB)	PSNR (dB)
1024	55.27	58.64	66.14
2048	58.95	61.03	70.32

Table 4. The comparison of the proposed radix-8 CORDIC design with some existing CORDIC-based designs.

References	FPGA used	Data width	Slice LUTs	Slice registers	Clock frequency
Xilinx CORDIC IP V.6 ³⁶	Virtex-7	16-bit	1201	1146	200 MHz
Aggarwal <i>et al.</i> ⁸	Virtex-5	16-bit	1301	205	202.065 MHz
Tang <i>et al.</i> ¹²	Kintex UltraScale	16-bit	300+5 DSPs	98	250 MHz
Mohamed <i>et al.</i> ¹³	Artix-7	16-bit	2202	645	276.6 MHz
Proposed	Virtex-7	16-bit	1185	759	363 MHz
Proposed	Virtex-7	20-bit	1512	1093	354 MHz
Proposed	Virtex-7	24-bit	1986	1458	338 MHz

is a saving of 31.71% FPGA resources in our case. The number of slice LUTs and the number of slices registered and consumed by the design proposed in Tang and Xu¹² is less. However, the design needs five DSPs for its FPGA implementation.¹² The area occupied by the DSPs and BRAMs in the case of the Xilinx Virtex-7 FPGA family is high. Each DSP block of the Virtex-7 FPGA family has an accumulator and a 25×18 multiplier and consumes an area approximately equivalent to 500 slices.⁴² Similarly, the block RAM occupies an area almost equal to 1,700 slices.⁴³

Table 5 shows a detailed comparison of the proposed design with some other existing designs. Our proposed design is compared with the other designs in three different data points, which are 16 points, 1024 points and 2048 points, respectively. There are many designs available in the literature that will use the dedicated block RAM and DSP of the FPGA. For the purpose of comparison, we have converted the blocks into equivalent slices. Here, the area utilization of the different designs is compared in terms of the number of slices. In the case of Nguyen *et al.*¹⁹ and Inguva

Table 5. Comparison of our FPGA-based FFT design with other designs.

No. of points	References	Data width	No. of LUTs	Block RAM	No. of DSPs	Total LUTs	Samples/cycle	Execution time (μ S)	Clock frequency
16	Kumar <i>et al.</i> ²⁷	16-bit	909	0	12	6,909	2	0.09	100 MHz
16	Bowlyn <i>et al.</i> ²¹	28-bit	38,072	0	0	38,072	16	—	75 MHz
16	Proposed	16-bit	3,209	0	0	3,209	4	0.04	323.4 MHz
1024	Ingemars. <i>et al.</i> ²⁵	16-bit	1,781	6	16	19,981	1	2.08	505 MHz
1024	Gamdo <i>et al.</i> ¹⁸	16-bit	1,425	0	48	25,425	4	1.06	270 MHz
1024	Nguyen <i>et al.</i> ¹⁹	16-bit	15,452	0	0	15,452	2	2.60	200 MHz
1024	Wang <i>et al.</i> ³¹	16-bit	8,678	10	45	39,678	4	1.75	305 MHz
1024	Kumar <i>et al.</i> ²⁷	16-bit	5282	0	62	36,282	2	6.39	100 MHz
1024	Proposed	16-bit	12,712	0	0	12,712	4	1.74	323.4 MHz
2048	Nguyen <i>et al.</i> ¹⁹	16-bit	19,414	0	0	19,414	2	5.17	200 MHz
2048	Wang <i>et al.</i> ³¹	16-bit	9,223	19	50	50,373	4	3.53	305 MHz
2048	Ayinala <i>et al.</i> ²⁹	16-bit	9,893	23	54	56,443	4	4.65	312 MHz
2048	Inguva <i>et al.</i> ²⁸	12-bit	19,242	0	0	19,242	2	5.5	200 MHz
2048	Proposed	16-bit	17,801	0	0	17,801	4	4.06	323.4 MHz

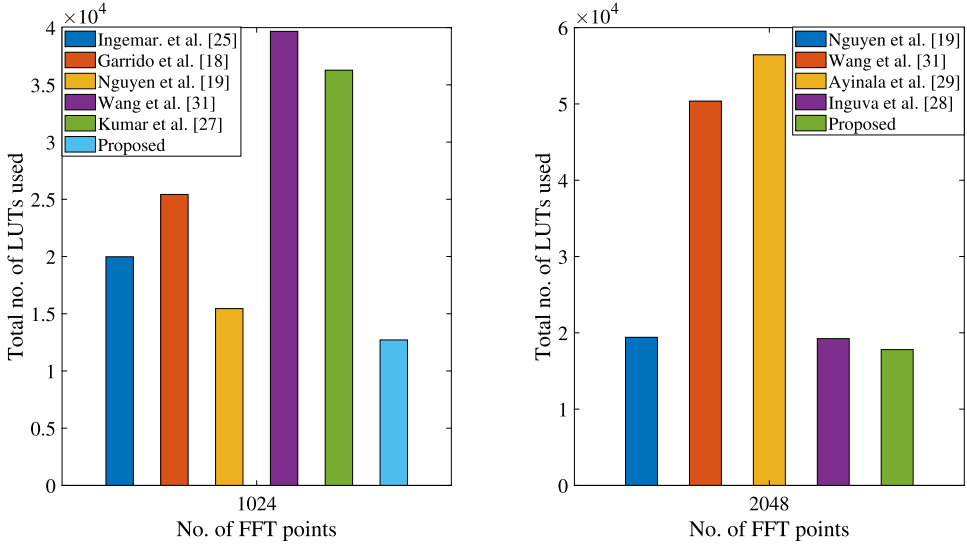


Fig. 15. Bar graph showing the hardware resources utilized for calculating 1024 and 2048 point FFT.

and Seventiline²⁸ the authors have used CORDIC to compute the butterfly structure. In Ayinala *et al.*,²⁹ Wang *et al.*,³¹ Kumar *et al.*²⁷ and Garrido *et al.*,¹⁸ the authors have used complex multipliers for the butterfly unit. However, in our case, we have used radix-8 CORDIC for the construction of butterfly unit.

For storing the twiddle angles in our case, we have used slice registers and memory LUTs. To compute the rotations, the proposed design took $m/3$ iterations for the m -bit precision. Whereas to compute the same task, the radix-2 CORDIC needs m iterations. This is one of the main reasons why the proposed design has less hardware utilization as compared to other proposed designs. For the 1024 points FFT, the execution time in Garrido *et al.*¹⁸ is less. However, the disadvantage of the design is the high area utilization. The FPGA resources utilized can be better analyzed from the bar graph shown in Fig. 15. Also, our design achieved a higher throughput of 1293.6 Mega samples per second (MSps).

The layout of the FFT is shown in Fig. 16. Two different Cadence tools are used for making the layout. By using the Cadence Genus tool, the code is synthesized and the cell library used is SCL 180 nm. After getting the synthesized netlist from the Genus tool, the netlist is imported to the Cadence Innovus tool to complete the PnR steps. In this work, the Mentor Calibre tool is used for the DRC, LVS and also for Antenna rule check.

Table 6 presents the specifications of the FFT ASICs designed with SCL 180 nm cell library. As shown in Table 6, the proposed FFT design is compared with the other two designs by considering the parameters core area, power dissipation, data width, clock rate, normalized energy, normalized area and execution time. The proposed design has the least normalized energy and area per FFT as compared to

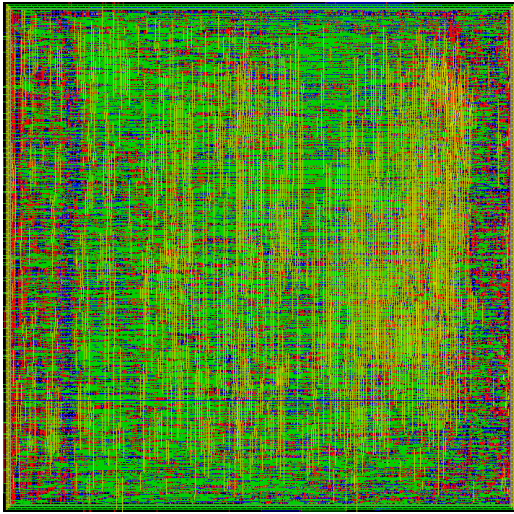


Fig. 16. Layout of the 1024 point radix-2² FFT.

the other mentioned FFT designs (normalized values are calculated using Eqs. (16) and (17), these equations are found in Ref. 26). As compared to the other FFT ASIC, our FFT ASIC can be operated at a higher clock frequency and has very less execution time. For the 2048 points FFT, the proposed design is 30.59 times faster as compared to the design proposed in Kang *et al.*²²

$$\text{Norm. Energy/FFT} = \frac{\text{Power} \times \text{clock cycles for execution}}{\text{Clock frequency} \times N \times \left(\frac{\text{Technology}}{180 \text{ nm}}\right) \times \left(\frac{V}{1.8}\right)^2}, \quad (16)$$

$$\text{Norm. Area/FFT} = \frac{\text{Core area}}{N \times \left(\frac{\text{Technology}}{180 \text{ nm}}\right)^2 \times \left[\frac{2}{3} \times \frac{WL}{24} + \frac{1}{3} \times \left(\frac{WL}{24}\right)^2\right]}. \quad (17)$$

The proposed ASIC implementation of the FFT is also compared with the state-of-the-art FFT ASIC designs by considering the same technology library. As compared to the method presented in Ref. 22, the proposed FFT ASIC has a little more

Table 6. Comparison of our ASIC-based FFT design with other designs.

References	22	26	Proposed	Proposed	Proposed	Proposed
Points (<i>N</i>)	2048	2048–16384	1024	2048	2048	2048
Cell library	110 nm	40 nm	180 nm	180 nm	110 nm	40 nm
Area of core	0.42 mm ²	2.358 mm ²	1.079 mm ²	1.544 mm ²	0.65 mm ²	0.32 mm ²
Power dissipation	3.64 mW	38.76 mW	81.32 mW	108.63 mW	57.00 mW	25.92 mW
Voltage	1.2 V	1.1 V	1.8 V	1.8 V	1.2 V	1.1 V
Clock frequency	50 MHz	150 MHz	285.7 MHz	285.7 MHz	298.33 MHz	315.05 MHz
Execution time	139.5 μS	27.81 μS	1.95 μS	4.56 μS	4.37 μS	4.14 μS
Norm. Energy/FFT	0.92 nJ	8.02 nJ	0.17 nJ	0.25 nJ	—	—
Norm. Area/ FFT(×10 ^{−3} mm ²)	1.34 mm ²	4.94 mm ²	1.05 mm ²	0.75 mm ²	—	—

core area and power dissipation. However, the design presented in Ref. 22 has a very high-execution time and less clock frequency than our design. The proposed design is 31.92 times faster than the design presented in Ref. 22. The execution time is the most crucial parameter in any real-time signal processing. The proposed design has the least execution time as compared to the other mentioned designs. The proposed design outperforms the design presented in Ref. 26 in all respects. As compared to Ref. 26, the proposed design has less core area and power dissipation. Also, the proposed design has much less execution time and high-clock frequency compared to the design presented in Ref. 26.

The hardware architecture design of FFT using FPGA and ASIC technology is pivotal in various industry sectors due to their distinct advantages. FPGAs offer flexibility and rapid prototyping, making them ideal for applications in wireless communication (such as orthogonal frequency-division multiplexing in 5G base stations), real-time audio and video processing, and medical imaging systems like MRI and CT scanners. On the other hand, ASICs provide high performance, which is crucial for high-volume production applications such as telecommunication satellites, high-performance computing (HPC) for scientific simulations, consumer electronics like DSP chips in smartphones and autonomous vehicles.

7. Conclusion

In this paper, we have presented hardware architecture design for the CORDIC-based radix-2² FFT. The proposed design's performance is evaluated in terms of computational speed, throughput, word length, hardware resource utilization, power dissipation and clock frequency. Our approach involved utilizing radix-8 CORDIC for constructing the butterfly units and replacing complex adders and multipliers. In this work, we have reduced the execution time and area of the design by lowering the number of rotational iterations and by precomputing the rotational directions in the CORDIC unit. The implemented radix-2² FFT has a simple architecture as that of a radix-2 FFT and can process four samples per clock cycle. When compared with the existing designs, the proposed design takes much less time to execute and uses fewer hardware resources. The design based on FPGA operates at a clock frequency of 323.4 MHz. The application specific integrated circuit has a small core area and can operate at a higher clock frequency. Due to the regularity of the proposed FFT architectures, the proposed design is well-suited for computing large-point FFT. In the future, by introducing the concept to higher radix FFT, execution time can be reduced further. In addition, better performance with minimal area consumption can be achieved by introducing pipelining in the higher radix FFT.

Acknowledgments

This work was financially supported by the SMDP-C2SD project sponsored by Meity, Government of India.

ORCID

Kaushik Das  <https://orcid.org/0000-0002-3467-1182>

Sambhu Nath Pradhan  <https://orcid.org/0000-0002-5461-6535>

Abhishek Bhattacharjee  <https://orcid.org/0000-0002-9664-1050>

References

1. K. R. Rao, D. N. Kim and J. J. Hwang, *Fast Fourier Transform: Algorithms and Applications* (New York, NY, 2010). doi:10.1007/978-1-4020-6629-0.
2. A. Cortes, I. Velez and J. F. Sevillano, Radix r^k FFTs: Matricial representation and SDC/SDF pipeline implementation, *IEEE Trans. Signal Process.* **57** (2009) 2824–2839. doi:10.1109/TSP.2009.2016276.
3. P. K. Meher, J. Valls, T. B. Juang, K. Sridharan and K. Maharatna, 50 years of CORDIC: Algorithms, architectures, and applications, *IEEE Trans. Circuits Syst. I: Regul. Pap.* **56** (2009) 1893–1907. doi:10.1109/TCSI.2009.2025803.
4. Y. Parmar and K. Sridharan, Precomputation-based radix-4 CORDIC for approximate rotations and Hough transform, *IET Circuits, Devices Syst.* **12** (2018) 413–423. doi:10.1049/iet-cds.2017.0492.
5. E. Antelo, J. Villalba, J. Bruguera and E. Zapata, High performance rotation architectures based on the radix-4 CORDIC algorithm, *IEEE Trans. Comput.* **46** (1997) 855–870. doi:10.1109/12.609275.
6. B. Lakshmi and A. S. Dhar, VLSI architecture for parallel radix-4 CORDIC, *Microprocess. Microsyst.* **37** (2013) 79–86. doi:10.1016/j.micpro.2012.12.001.
7. P. Choudhary and A. Karmakar, CORDIC based implementation of fast fourier transform, *2nd Int. Conf. Computer and Communication Technology*, Allahabad, India, 2011, pp. 550–555, doi:10.1109/ICCCT.2011.6075128.
8. S. Aggarwal, P. K. Meher and K. Khare, Concept, design, and implementation of reconfigurable CORDIC, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **24** (2016) 1588–1592. doi:10.1109/TVLSI.2015.2445855.
9. R. Shukla and K. C. Ray, Low latency hybrid CORDIC algorithm, *IEEE Trans. Comput.* **63** (2014) 3066–3078. doi:10.1109/TC.2013.173.
10. A. Changela, M. Zaver and A. Lakhani, ASIC implementation of high performance radix-8 CORDIC algorithm, *Int. Conf. Advances in Computing, Communications and Informatics* (IEEE, 2018), pp. 699–705. doi:10.1109/ICACCI.2018.8554883.
11. S. F. Obermann and M. J. Flynn, Division algorithms and implementations, *IEEE Trans. Comput.* **46** (1997) 833–854. doi:10.1109/12.609274.
12. W. Tang and F. Xu, A noniterative radix-8 CORDIC algorithm with low latency and high efficiency, *Electronics* **9** (2020) 1521. doi:10.3390/electronics9091521.
13. S. M. Mohamed, W. S. Sayed, A. G. Radwan and L. A. Said, FPGA implementation of reconfigurable CORDIC algorithm and a memristive chaotic system with transcendental nonlinearities, *IEEE Trans. Circuits Syst. I: Regul. Pap.* **69** (2022) 2885–2892. doi:10.1109/TCSI.2022.3165469.
14. C. Liu, Aygn, K. and Ylmaz, A., A parallel FFT-accelerated layered-medium integral-equation solver for electronic packages, *Int. J. Numer. Model.: Electron. Netw. Devices Fields* **33** (2019) 1–17. doi:10.1002/jnm.2684.
15. M. A. El-Motaz, O. A. Nasr and K. Osama, A CORDIC-friendly FFT architecture, *Int. Wireless Communications and Mobile Computing Conf.*, Nicosia, Cyprus, 2014, pp. 1087–1092. doi:10.1109/IWCMC.2014.6906506.

16. K. Das and S. N. Pradhan, Hardware architecture design for complementary ensemble empirical mode decomposition algorithm, *Integration* **91** (2023) 153–164. doi:10.1016/j.vlsi.2023.03.011.
17. Z. Abo-Hammour, O. Abu Arqub, S. Momani and N. Shawagfeh, Optimization solution of Troesch's and Bratu's problems of ordinary type using novel continuous genetic algorithm, *Discret. Dyn. Nat. Soc.* **2014** (2014) 1–15. doi:10.1155/2014/401696.
18. M. Garrido, J. Grajal, M. A. Sanchez and O. Gustafsson, Pipelined radix-2^k feedforward FFT architectures, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **21** (2013) 23–32. doi:10.1109/TVLSI.2011.2178275.
19. N. H. Nguyen, S. A. Khan, C. H. Kim and J. M. Kim, A high-performance, resource-efficient, reconfigurable parallel-pipelined FFT processor for FPGA platforms, *Microprocess. Microsyst.* **60** (2018) 96–106. doi:10.1016/j.micpro.2018.04.003.
20. S. N. Tang, J. W. Tsai and T. Y. Chang, A 2.4-GS/s FFT processor for OFDM-based WPAN applications, *IEEE Trans. Circuits Syst. I: Regul. Pap.* **57** (2010) 451–455. doi:10.1109/TCSII.2010.2048373.
21. K. Bowlyn and S. Hounsinou, An improved distributed multiplier-less approach for radix-2 FFT, *IEEE Lett. Comput. Soc.* **3** (2020) 54–57. doi:10.1109/LOCS.2020.3014354.
22. G. Kang, W. Choi and J. Park, Embedded DRAM-based memory customization for low-cost FFT processor design, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **25** (2017) 3484–3494. doi:10.1109/TVLSI.2017.2752265.
23. X. Shih, H. Chou and Y. Liu, VLSI design and implementation of reconfigurable 46-mode combined-radix-based FFT hardware architecture for 3GPP-LTE applications, *IEEE Trans. Circuits Syst. I: Regul. Pap.* **65** (2018) 118–129. doi:10.1109/TCSI.2017.2725338.
24. S. M. Noor, E. John and M. Panday, Design and implementation of an ultralow-energy FFT ASIC for processing ECG in cardiac pacemakers, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **27** (2019) 983–987. doi:10.1109/TVLSI.2018.2883642.
25. C. Ingemarsson and O. Gustafsson, SFF — The single-stream FPGA-optimized feed-forward FFT hardware architecture, *J. Signal Process. Syst.* **90** (2018) 1583–1592. doi:10.1007/s11265-018-1370-y.
26. J. Wang, S. Li and X. Li, Scheduling of data access for the Radix-2k FFT processor using single-port memory, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **28** (2020) 1676–1689. doi:10.1109/TVLSI.2020.2992021.
27. G. P. Kumar, B. T. Krishna and K. Pushpa, Optimized pipelined fast fourier transform using split and merge parallel processing units for OFDM, *Wirel. Pers. Commun.* **117** (2021) 3067–3089. doi:10.1007/s11277-020-07471-3.
28. S. C. Inguva and J. B. Seventiline, Implementation of FPGA design of FFT architecture based on CORDIC algorithm, *Int. J. Electron.* **108** (2021) 1914–1939. doi:10.1080/00207217.2020.1870750.
29. M. Ayinala, M. Brown and K. K. Parhi, Pipelined parallel FFT architectures via folding transformation, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **20** (2012) 1068–1081. doi:10.1109/TVLSI.2011.2147338.
30. Z. Wang, X. Liu, B. He and F. Yu, A combined SDC-SDF architecture for normal i/o pipelined radix-2 FFT, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **23** (2015) 973–977. doi:10.1109/TVLSI.2014.2319335.
31. J. Wang, C. Xiong, K. Zhang and J. Wei, A mixed-decimation MDF architecture for radix-2k parallel FFT, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **24** (2016) 67–78. doi:10.1109/TVLSI.2015.2402207.
32. Y. Guo, Z. Wang, Q. Hong, H. Luo, X. Qiu and L. Liang, A 60-mode high-throughput parallel-processing FFT processor for 5G/4G applications, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **31** (2023) 219–232. doi:10.1109/TVLSI.2022.3227346.

33. J. Hazarika, S. R. Ahamed and H. B. Nemade, Low-complexity, energy-efficient fully parallel split-radix FFT architecture, *Electron. Lett.* **58** (2022) 678–680. doi:10.1109/ISCAS.2003.1205774.
34. A. X. G. X. Chelliah, A normal I/O order optimized dual-mode pipelined FFT architecture for processing real-valued signals and complex-valued signals, *AEU-Int. J. Electron. Commun.* **170** (2023) 154782. doi:10.1016/j.aeue.2023.154782.
35. H. Fang, Z. Ma, F. Yu, B. Zhao and B. Zhang, Optimised serial commutator FFT architecture in terms of multiplexers, *IEEE Trans. Circuits Syst. II: Express Br.* **71**(1) (2024) 445–449. doi:10.1109/TCSII.2023.3304696.
36. K. Das, E. H. Reddy, A. Mandal, S. N. Pradhan and A. Bhattacharjee, Area-time-energy efficient architecture of CBNS-based fast fourier transform, *Int. J. Circuit Theory Appl.* (2024). doi:10.1002/cta.4075.
37. S. He and M. Torkelson, A new approach to pipeline FFT processor, *Proc. Int. Conf. Parallel Processing*, Honolulu, HI, USA, 1996, pp. 766–770. doi:10.5555/645606.661012.
38. M. Kuhlmann and K. K. Parhi, P-CORDIC: A precomputation based rotation CORDIC algorithm, *Appl. Signal Process.* **2002** (2002) 936–943. doi:10.1155/S1110865702205028.
39. K. K. Parhi, *VLSI Digital Signal Processing Systems* (John Wiley and Sons, 1999).
40. G. A. Ruiz and M. Granda, Efficient canonic signed digit recoding, *Microelectron. J.* **42** (2011) 1090–1097. doi:10.1016/j.mejo.2011.06.006.
41. Xilinx Inc., LogiCORE IP CORDIC v6.0, product specifications IPPG105 (2017).
42. Xilinx Inc., User guide for 7 series DSP48E1 slice (2014). Available at: http://www.xilinx.com/support/documentation/user_guides/ug479_7Series_DSP48E1.pdf.
43. Xilinx Inc., User guide for 7 series FPGAs memory resources (2014). Available at: http://www.xilinx.com/support/documentation/user_guides/ug473_7Series_Memory_Resources.pdf.